



Silesian
University
of Technology

MASTER THESIS

Applying AI Algorithms to Train a Computer Player in a Logic Board Game

Jagoda Barbara WIĘCEK

Student identification number: 〈jw300503〉

Programme: Control, Electronic and Information Engineering

Specialisation: Data Science

SUPERVISOR

〈Dr Ewa Lach〉

**DEPARTMENT 〈Department of Graphics, Computer Vision and
Digital Systems〉**

Faculty of Automatic Control, Electronics and Computer Science

Gliwice 2026

Thesis title

Applying AI Algorithms to Train a Computer Player in a Logic Board Game

Abstract

(Thesis abstract – to be copied into an appropriate field during an electronic submission – in English.)

Key words

(2-5 keywords, separated by commas)

Tytuł pracy

Zastosowanie algorytmów sztucznej inteligencji do nauki komputerowego gracza w logicznej grze planszowej

Streszczenie

(Streszczenie pracy – odpowiednie pole w systemie APD powinno zawierać kopię tego streszczenia.)

Słowa kluczowe

(2-5 słów (fraz) kluczowych, oddzielonych przecinkami)

Contents

1	Introduction	1
1.1	Introduction into the problem domain	1
1.2	Settling of the problem in the domain	2
1.3	Objective of the thesis	3
1.4	Scope of the thesis	3
1.5	Short description of chapters	4
1.6	Description of contribution of the thesis's author	4
2	Analysis of the problem	5
2.1	Problem statement	5
2.2	State of the art	5
2.2.1	An Overview of Non-Player Character Behavior Systems	6
2.2.2	Deep Learning	10
2.2.3	Reinforcement Learning	12
2.2.4	Policy Gradient Methods	14
2.2.5	Proximal Policy Optimization	14
2.2.6	Soft Actor-Critic	14
2.3	Literature Research	14
2.4	Existing solutions	14
3	Subject of the thesis	15
3.1	[Section title]	15
3.2	[Subsection title]	15
4	Experiments	17
5	Summary	19
	Bibliography	22
	Technical documentation	25
	List of abbreviations and symbols	27

List of additional files in electronic submission (if applicable)	29
List of figures	31
List of tables	33

Chapter 1

Introduction

Board games have entertained people since ancient times, from teaching techniques for the youngest, to providing entertainment for entire families, to competitions for the best players. Nowadays, many popular board games have been introduced into the digital world to bring people together from different corners of the globe. The digital board game environment has also made it possible to play games without the participation of physical people on the other side of the screen. The implemented players were tasked with behaving like human participants.

This thesis focuses on applying AI algorithms to train a computer player for a logic-based board game. The selected game introduces distinct challenges, including combinatorial complexity, dynamic board states, randomness and the necessity for both tactical and strategic reasoning. Designing an AI capable of playing effectively in such an environment requires balancing rule-based knowledge with adaptive learning techniques

1.1 Introduction into the problem domain

The history of board games spans thousands of years, reflecting their enduring role in human culture. The Royal Game of Ur, originating in ancient Mesopotamia more than 4,500 years ago, is considered one of the earliest known board games [2]. Since then, board games have remained a popular pastime across civilizations, valued not only as a form of entertainment but also as a means of social interaction and intellectual challenge.

Beyond recreation, board games have also been recognised for their pedagogical value. Recent studies highlight their efficacy as educational tools, showing that board games can enhance motivation, engagement, and learning outcomes in classroom settings [6]. By providing structured environments that require logical thinking, planning, and decision-making, board games naturally support the development of cognitive and social skills.

Board games have long been used as a way to test logical thinking and strategy. Unlike purely abstract games such as chess, many modern tactical board games introduce an element of randomness, such as shuffled decks, dice rolls, or random tile draws. This

combination of determinism and uncertainty contributes to a more complex decision-making process. Players are required to plan tactics based on the current position, whilst also having the capacity to adapt to unpredictable events.

The dual role of board games as both an entertainment medium and an educational tool renders them an ideal domain for exploring artificial intelligence algorithms. These games combine structured rule systems with varying levels of complexity, randomness, and strategic depth. Consequently, the training of a computer player for such games contributes not only to the field of game AI but also offers broader insights into decision-making processes, learning under constraints, and adaptive strategies in uncertain environments.

1.2 Settling of the problem in the domain

Many popular board games that use artificial intelligence algorithms only required learning the rules and adapting tactics to the opponent. One such game is chess, which has many artificial intelligence algorithms implemented, one of which is **Stockfish - Open Source Chess Engine** [3]. However, in numerous games, players are required to adjust their tactics not only to their opponents' actions, but also to environmental and situational randomness. Rummikub exemplifies this, requiring players to adapt their strategy to other players, the random draw of tiles, and the specific rules for laying down sets.

In Rummikub, players create and rearrange sets and sequences of numbered tiles in different colors. The objective of the game is to place all tiles from one's hand onto the shared board by forming valid combinations according to the game's rules. However, in contrast to purely deterministic games, Rummikub introduces stochastic elements, such as random tile draws or randomly generated tiles at the beginning of the game. These elements have been shown to significantly affect the course of the game. This randomness forces both human and artificial players to adapt their tactics dynamically to ever-changing conditions on the board.

From an AI perspective, this environment poses several challenges. As the board evolves, the number of potential moves increases, rendering an exhaustive search impractical. The uncertainty of future tile draws and opponent actions introduce an additional layer of complexity that requires probabilistic reasoning or learning-based adaptation rather than deterministic planning alone.

Positioning this problem within the broader field of game AI highlights its research value. Developing an effective computer player for Rummikub requires combining strategic evaluation, pattern recognition, and adaptive learning. It also provides an opportunity to investigate how modern AI algorithms, such as reinforcement learning and simulation-based decision-making, can operate effectively in environments that are logical in structure but influenced by randomness.

1.3 Objective of the thesis

The main objective of this thesis is to analyse, evaluate, and experimentally verify whether selected artificial intelligence algorithms can effectively support decision-making of a computer-controlled player in the board game Rummikub. The developed AI system should be able to analyze the current game state, evaluate available legal moves, and select actions that maximize its probability of success in a random environment.

Due to the stochastic nature of Rummikub, particularly the randomness associated with tile distribution and draws, the computer player must be able to operate in a dynamic and partially unpredictable environment. Therefore, a key objective of this work is to propose AI algorithms and their parameters that can adapt their strategy to changing board configurations, varying hand compositions and evolving tactical opportunities throughout the game.

1.4 Scope of the thesis

This thesis focuses on the application of Reinforced Learning algorithms to train a computer player for the board game Rummikub within a custom-developed digital environment. The scope of the work includes the design and implementation of the game logic, the integration of Non-Player Character (NPC), and the experimental evaluation of the trained computer player. The project is limited to a single board game, Rummikub, and does not attempt to generalise the developed solution to other game genres. The research excludes online multiplayer deployment and human-subject studies. The implementation is carried out using the Unity engine, where the game environment and computer players are developed and tested.

The thesis maintains focus on the core research problem: the practical application of AI algorithms to enable intelligent decision-making in a tactical board game environment characterised by combinatorial complexity and stochastic elements. The development of the project include:

- Implementation of game environment for the player - development of a digital representation of the Rummikub board game using the Unity engine, including tile management, valid set detection, and interaction rules.
- Integrating artificial intelligence algorithm - implementation of the different reinforcement learning algorithms available from Unity ML-Agents Toolkit.
- Developing a system of rewards and penalties - implementation of properly working points system for better training the model.
- Training the NPC - training of the computer player based on simulated games, allowing it to improve its tactics through experience and feedback.

- Evaluating the model - conduction of the experiments to evaluate the efficiency, adaptability and success rate of the trained computer player.

1.5 Short description of chapters

TODO at the end of whole thesis

1.6 Description of contribution of the thesis's author

The essential elements of the project, that include the author contributions, are:

- Development of experiments environment - the design and implementation of a fully functional digital Rummikub environment, including game logic, rule validation, and agent interaction mechanisms.
- Supervision and configuration of the learning process – the definition of the state and action spaces, design of the reward structure, configuration of training parameters, and monitoring of the computer player's learning progress.
- Evaluation of the computer player's performance – the design and execution of simulation-based experiments to assess the effectiveness, adaptability, and decision-making quality of the trained AI model.
- Analysis and interpretation of results – the examination of experimental outcomes and formulation of conclusions regarding the applicability of artificial intelligence methods to tactical board games with stochastic elements.

Chapter 2

Analysis of the problem

This chapter presents a detailed analysis of the research problem addressed in this thesis. The objective of this work is to design and evaluate artificial intelligence algorithms capable of training a computer-controlled player in a logic-based board game environment. The developed 2.5D board game serves as an experimental platform for testing different learning approaches.

2.1 Problem statement

The development of intelligent NPC capable of operating effectively in complex, rule-based board games remains a significant challenge in artificial intelligence. While classical board games such as chess and Go have been successfully addressed using advanced reinforcement learning and search-based techniques, many logic-based tile placement games introduce additional layers of combinatorial complexity, dynamic action spaces, and stochastic variability.

This thesis addresses the problem of training a computer-controlled player in a complex logic board game implemented in the Unity environment. The game serves as an experimental testbed for evaluating artificial intelligence algorithms under conditions of:

- high-dimensional state representation,
- large and dynamically changing action space,
- strict rule-based constraints,
- significant randomness in gameplay.

2.2 State of the art

making algorithm has played a significant role in the domain of digital games for several decades. From early deterministic rule-based systems to modern deep learning architec-

tures, AI algorithms have continuously evolved to enhance the behavior of Non-Player Characters (NPCs) and automated opponents. Over time, various algorithmic approaches have been employed, including finite state machines, heuristic search methods, planning algorithms, and, more recently, reinforcement learning techniques. This section reviews the most relevant contemporary approaches to artificial intelligence algorithms used in games, with particular emphasis on reinforcement learning algorithms and decision-making systems applicable to complex, rule-based board games with large and dynamic action spaces.

2.2.1 An Overview of Non-Player Character Behavior Systems

According to [9], Artificial Intelligence (AI) can be defined as simulated intelligence on programmable machines that attempts to mimic the human brain.

It is a thriving field, with active research topics aimed at automating routine labor, understanding speech and images, and supporting basic scientific research. AI systems are designed to solve tasks that are intellectually challenging for humans, but can be described formally (e.g. playing chess), as well as intuitive tasks that are difficult to describe formally (e.g. recognising faces).

The aim of applying artificial intelligence algorithms in digital games is to create computer-controlled players that can imitate human behavior and compete effectively with human opponents. A key challenge is designing a computer player that not only performs well and is capable of winning against human players, but also demonstrates behavior that appears natural and realistic during gameplay. To achieve this, developers and researchers employ a variety of AI algorithms and computational models:

- Finite State Machines (FSMs)

It is a foundational, abstract computational model utilized extensively in video game development to govern the decision-making processes of NPCs, operating on the principle that an entity can only exist in one of several predefined states at any given moment. Despite the emergence of more advanced AI architectures, FSMs remain highly relevant in modern game development, functioning either as standalone systems or as complementary modules in complex AI frameworks. An FSM is formally defined by a 5-tuple:

$$M = (\Sigma, S, s_0, \delta, F) \tag{2.1}$$

Where

- Σ is the input alphabet, finite set of input signals
- S is a finite set of states
- s_0 is the starting state

- δ is the state-transition function
- F represents final states

While highly efficient, FSMs suffer from scalability issues, as an increase in states can lead to an exponential explosion of transition rules [13].

- Behavior Trees (BTs)

BTs solve the scalability issues of FSMs by utilizing a hierarchical, directed acyclic graph structure. The architecture consists of root nodes, control flow nodes (composites like Sequences and Selectors), decorator nodes, and execution leaves (actions). The system operates via a signal called a "tick" that traverses the tree at regular intervals [13]. The execution of a single sequence proceeds as follows:

1. The Sequence node receives an execution tick from its parent.
2. The node begins executing its child nodes sequentially from left to right.
3. If the child returns a *Success* state, proceed to execute the next child in the sequence.
If the child returns a *Running* state, halt execution and return *Running* to the parent.
If the child returns a *Failure* state, halt execution and immediately return *Failure* to the parent.
4. If all child nodes have been executed and returned *Success*, the Sequence node returns *Success* to its parent

- Rule-Based Systems (RBS)

They represent a classical decision-making paradigm within artificial intelligence. An RBS is structurally segregated into three primary components: an information database, a formalized rule set, and an arbiter mechanism. The database acts as the computer player's memory, storing environmental observations and internal states. The rule set contains multiple declarative conditional statements (if-then-else rules). The arbiter acts as the central processing unit, analyzing the database against the rule set to trigger the appropriate action. [13]

- Heuristic Search

These algorithms are systematic exploration techniques utilized to navigate through vast state spaces efficiently by incorporating domain-specific knowledge. Unlike blind search methods, heuristic algorithms evaluate the potential of intermediate states to reach a goal, drastically reducing the computational overhead. In modern game architectures, such as hyper-heuristics, heuristic search is frequently deployed for high-level macro-navigation or to solve deterministic portions of a game, while other algorithms take over for stochastic or localized maneuvering. [10]

- Monte Carlo Tree Search (MCTS)

This algorithm is a sophisticated heuristic search algorithm engineered to solve sequential decision-making problems within environments characterized by extraordinarily large and complex state spaces. The fundamental architecture of MCTS synthesizes the precision of classical tree search methodologies with the stochastic nature of the Monte Carlo method, which is a computational technique that uses repeated random sampling to obtain numerical results for complex problems. By simulating systems thousands or millions of times with random inputs, it estimates probabilities and outcomes [4]. MCTS dynamically and incrementally constructs a search tree in memory. In this tree structure, each node represents a specific game state, while the connecting edges correspond to the legal actions that transition the environment from one state to another. [1], [5]. The algorithm goes as follows

1. Instantiate the root node S_0 representing the current environment state.
2. While the predefined computational budget is not exhausted, begin a new search iteration
3. Traverse the tree from S_0 Upper Confidence Bound policy until a leaf node S_L is reached
4. if S_L is not terminal state, generate a new child node S_{new} by applying a legal action to S_L
5. Execute a rollout from S_{new} using a default randomized policy until a terminal state S_T is reached
6. Compute the reward R of the terminal state S_T
7. Update the visit counts N and the accumulated rewards W for all nodes along the path from S_{new} back to S_0 using the value R
8. Once the budget is depleted, select the child of the root node S_0 with the highest visit count as the optimal action to execute in the real environment

- Minimax Algorithm

This algorithm is a classical recursive strategy utilized predominantly in zero-sum, complete-information board games. It models a strictly adversarial environment where one agent seeks to maximize its utility while the opponent aims to minimize it. The algorithm constructs a game tree representing all possible future moves. Since exhaustive search is practically impossible for complex games due to state space explosion, the search is truncated at a specific depth, and a manually designed heuristic evaluation function is used to score the leaf nodes [11]. The algorithm goes

as follows

$$v(s) = \begin{cases} eval(s) & \text{if } s \text{ is terminal} \\ \max_a v(result(s, a)) & \text{if maximizing player} \\ \min_a v(result(s, a)) & \text{if minimizing player} \end{cases} \quad (2.2)$$

Where

- v is value of state, which estimates how favorable a given situation is for the player
- s is the current state of game, representing a specific configuration of all game elements at a given time.
- a is an action that player can make in state s to transition to a new state

The algorithm with depth limit

1. Call the recursive function with the current state, current depth, and the maximizing player's identifier.
 2. If the maximum depth is reached or the state is terminal (win/loss/draw), invoke the static evaluation function to score the board and return the value.
 3. If the current agent is the maximizing player, initialize the best value to negative infinity. Iterate through all possible legal moves. For each move, recursively call the Minimax algorithm. Update the best value to the maximum of the current best value and the recursive result.
 4. If the current agent is the minimizing player (opponent), initialize the best value to positive infinity. Iterate through all legal moves. Recursively call the algorithm, updating the best value to the minimum of the current best value and the recursive result.
 5. Return the computed optimal value up the recursive stack to determine the best immediate move at the root level.
- Genetic Algorithms (GA)
- These algorithms represent a robust category of stochastic optimization and search techniques heavily inspired by the biological principles of natural selection and evolution. In the context of computer games, these algorithms do not rely on traditional numerical expressions but operate on populations of candidate solutions, often encoded as chromosomes. The evaluation of each individual in the population is handled by an adaptation function, which calculates a numerical assessment known as fitness [7]. This algorithm goes as follows

1. Generate an initial population of random individuals.
 2. Assess the adaptation value (fitness) of every individual in the current population using the game's simulation environment.
 3. Select the fittest individuals to act as parents for the next generation.
 4. Merge segments of the parent chromosomes to produce new offspring.
 5. Introduce random alterations to certain genes within the offspring to maintain genetic diversity and prevent premature convergence.
 6. Replace the old population with the newly generated offspring and repeat steps 2-5 until a termination condition is satisfied.
- Reinforcement Learning (RL)
Algorithm allows agents to learn optimal policies through trial-and-error interactions using reward-penalty system. This topic is described in detail in chapter 2.2.3.

2.2.2 Deep Learning

History of Deep Learning

The pursuit of artificial intelligence dates back to ancient times, but, according to [4], as a computational discipline it has evolved through three waves:

1. The First Wave: Cybernetics (1940s–1960s):

The earliest computational models of learning were intended to be biological models of how learning happens in the brain. This era, known as cybernetics, focused on biologically inspired linear models. In 1943, the McCulloch–Pitts neuron was introduced as an early model of brain function, capable of recognising two categories by testing whether a linear combination of inputs was positive or negative. In the 1950s, this was followed by the Perceptron (Rosenblatt, 1958), which was the first model capable of automatically learning the weights that define categories from given examples. Around the same time, the Adaptive Linear Element (ADALINE) was developed to predict real numbers using an early form of stochastic gradient descent. However, this first wave ended in a severe backlash. Critics, most notably Minsky and Papert in 1969, demonstrated the profound limitations of linear models. Famously, they proved that simple linear models could not learn the XOR (exclusive OR) logic function. This mathematical limitation caused a massive decline in neural network research funding and popularity.

2. The Second Wave: Connectionism (1980s–1990s)

The second wave emerged in the context of cognitive science under the movement known as connectionism or parallel distributed processing. The central thesis of

connectionism was that intelligent behavior emerges from a large number of simple computational units networked together. This era introduced two monumental concepts that remain central to modern deep learning:

- Distributed Representations - Introduced by Hinton et al. (1986), this is the principle that each input to a system should be represented by many features, and each feature should be involved in representing many possible inputs
- Back-propagation - The successful use and popularization of the back-propagation algorithm by Rumelhart et al. (1986) allowed for the training of nowadays neural networks with multiple hidden layers.

Despite introducing the mathematical foundations used today, this wave faded by the mid-1990s. Unrealistic claims were made about AI capabilities, leading to investor disappointment when researchers could not deliver. Furthermore, other machine learning paradigms with convex objective functions, such as Kernel machines (Support Vector Machines) and graphical models, began to outperform neural networks on important tasks.

3. The Third Wave: Deep Learning (2006–Present)

The modern renaissance of neural networks began in 2006, overcoming the long-held belief that deep models (models with many hidden layers) were too difficult to optimize. The breakthrough was led by Geoffrey Hinton, who demonstrated that a Deep Belief Network (DBN) could be efficiently trained using a strategy called greedy layer-wise unsupervised pretraining. The concept of this training procedure can be described through the following sequence of steps:

- Define a deep neural network architecture with multiple hidden layers.
- Train the first hidden layer in isolation using an unsupervised algorithm (like a Restricted Boltzmann Machine or an Autoencoder) to capture the shape and representation of the raw input data.
- Once the first layer is trained, pass the raw input data through it to extract a new, intermediate representation of the data.
- Use the intermediate representation from the previous layer as the input to train the next hidden layer in isolation.
- Repeat steps 3 and 4 for all hidden layers in the network.
- After all layers are pretrained, add a classification layer on top and use the back-propagation algorithm to jointly train (fine-tune) the entire network using labeled data.

This discovery proved that deep architectures could outperform shallow models like Support Vector Machines (e.g., on the MNIST dataset). This wave popularized the specific term "Deep Learning" to emphasize the theoretical importance of depth in architectural design.

Today, while the foundational algorithms like back-propagation remain the same as in the 1980s, the current wave's explosive success is heavily attributed to the age of "Big Data" (providing the millions of labeled examples necessary for statistical generalization) and the massive parallel computational power of modern general-purpose GPUs

Algorithm of Deep Learning

2.2.3 Reinforcement Learning

History of Reinforcement Learning

In 1950 Alan Turing published a paper, "Computing machinery and intelligence" [12], where he stated a crucial question: "Can machines think?". Alan Turing presented a game called **Imitation Game**, known nowadays as **Turing Test**, that proceeded as follows:

1. Set up a remote scenario involving three participants: a machine programmed to imitate a human (Player A), a human (Player B) and an interrogator (Player C), who can be either male or female.
2. Isolate the interrogator (Player C) in a separate room. Ensure that all communication is via a text-only medium to eliminate physical, visual and vocal biases.
3. Player A (the machine) receives questions from Player C and processes them to generate responses that deliberately mislead the interrogator. To simulate human fallibility, it may introduce typing pauses or even mathematical errors.
4. Player B provides truthful answers and attempts to assist Player C in correctly identifying the machine.
5. Player C must evaluate the text responses and declare "X is A and Y is B" or vice versa. If the interrogator decides wrongly as often as they would in a standard man-woman iteration of the game, the machine is deemed to exhibit satisfactory intellectual behavior.

Turing correctly identifies that manually coding an exhaustive "table of instructions" to simulate an adult human mind is a computationally impossible task. His proposed paradigm shift is the conceptual birth of Machine Learning. Instead of programming an adult, engineers should simulate a "child brain", a system with minimal initial mechanisms

and vast amounts of blank storage. This system is then subjected to an educational regimen. Turing explicitly draws a mathematical and systemic parallel to biological evolution:

- Structure of the child machine = hereditary material
- Changes of the child machine = mutation
- Natural selection = judgment of the experimenter

The Child Machine Learning Loop algorithm is proceeded as follows:

1. Instantiate a discrete-state machine with a minimal instruction set and an integrated "random element" generator
2. The experimenter provides a task, instruction, or data via unemotional communication channels (like a symbolic language)
3. The machine generates an output. The random element allows for non-systematic exploration of the solution space, avoiding infinite loops
4.
 - If the output is incorrect, apply a punishment signal. The machine's internal parameters are updated to decrease the probability of repeating the preceding sequence
 - If the output is correct, apply a reward signal. The parameters are updated to increase the probability of sequence repetition
5. The rules of operation (the program) dynamically adapt over time. The machine evolves from a blank slate to a highly complex cognitive engine capable of competing in intellectual fields like chess or natural language processing

By using trial, error, rewards, and punishments, the child machine slowly writes its own adult rulebook. This idea represents an early conceptual foundation of what is now known as reinforcement learning.

Fundamentals of Reinforcement Learning

Deep Reinforcement Learning

2.2.4 Policy Gradient Methods

2.2.5 Proximal Policy Optimization

2.2.6 Soft Actor-Critic

2.3 Literature Research

2.4 Existing solutions

The development of computer players for board games has been a significant research area within the field of artificial intelligence for many years. Numerous studies have explored different approaches to decision-making, strategy optimization, and learning in game environments.

Chapter 3

Subject of the thesis

tekst

3.1 [Section title]

3.2 [Subsection title]

Each figure in the document should be referred to at least once (fig. 3.1).

Each table in the document should be referred to at least once (Tab. 3.1).

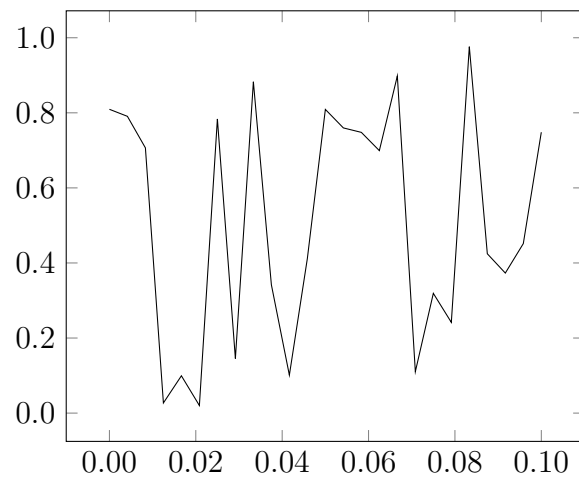


Figure 3.1: Figure caption.

Table 3.1: A caption of a table is ABOVE it.

ζ	method						
	alg. 1	alg. 2	alg. 3			alg. 4, $\gamma = 2$	
			$\alpha = 1.5$	$\alpha = 2$	$\alpha = 3$	$\beta = 0.1$	$\beta = -0.1$
0	8.3250	1.45305	7.5791	14.8517	20.0028	1.16396	1.1365
5	0.6111	2.27126	6.9952	13.8560	18.6064	1.18659	1.1630
10	11.6126	2.69218	6.2520	12.5202	16.8278	1.23180	1.2045
15	0.5665	2.95046	5.7753	11.4588	15.4837	1.25131	1.2614
20	15.8728	3.07225	5.3071	10.3935	13.8738	1.25307	1.2217
25	0.9791	3.19034	5.4575	9.9533	13.0721	1.27104	1.2640
30	2.0228	3.27474	5.7461	9.7164	12.2637	1.33404	1.3209
35	13.4210	3.36086	6.6735	10.0442	12.0270	1.35385	1.3059
40	13.2226	3.36420	7.7248	10.4495	12.0379	1.34919	1.2768
45	12.8445	3.47436	8.5539	10.8552	12.2773	1.42303	1.4362
50	12.9245	3.58228	9.2702	11.2183	12.3990	1.40922	1.3724

Chapter 4

Experiments

Chapter 5

Summary

- synthetic description of performed work
- conclusions
- future development, potential future research
- Has the objective been reached?

Bibliography

- [1] Tina Babu, Tejaswi S, Anup S B, Srinivas, Narla Rama Krishna Reddy, Manasvi S and Dhiraj Singh. ‘Monte Carlo Tree Search Optimization for Go Game AI’. In: *2025 International Conference on Emerging Technologies in Computing and Communication (ETCC)*. 2025, pp. 1–6. DOI: 10.1109/ETCC65847.2025.11108437.
- [2] ‘Board Games’. eng. In: *The Biblical archaeology review* 49.1 (2023), pp. 26–. ISSN: 0098-9444.
- [3] Stockfish Developers. *Stockfish - Open Source Chess Engine*. 2024. URL: <https://github.com/official-stockfish/Stockfish> (visited on 22/12/2024).
- [4] Ian Goodfellow, Yoshua Bengio and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [5] Makoto Ishihara, Taichi Miyazaki, Chun Yin Chu, Tomohiro Harada and Ruck Thawonmas. ‘Applying and Improving Monte-Carlo Tree Search in a Fighting Game AI’. In: *Proceedings of the 13th International Conference on Advances in Computer Entertainment Technology*. ACE ’16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 1–6. DOI: 10.1145/3001773.3001797. URL: <https://doi.org/10.1145/3001773.3001797>.
- [6] Claudia Jipa, Carmen Alina Berce and Mirela Cosma. ‘The impact of using board games on the development of situational motivation in primary school children’. eng. In: *Educația-plus* 33.Special Issue (2023), pp. 10–27. ISSN: 1842-077X.
- [7] Marek Kopel and Adam Pocijowski. ‘Applying ML Algorithms to Video Game AI’. In: *Intelligent Information and Database Systems*. Vol. 11683. Lecture Notes in Artificial Intelligence. Springer Nature Switzerland AG, 2019, pp. 353–362. DOI: 10.1007/978-3-030-28377-3_29. URL: https://doi.org/10.1007/978-3-030-28377-3_29.
- [8] M&M Ventures (2014) Limited. *Rummikub Rules (English)*. 2019. URL: <https://rummikub.com/wp-content/uploads/2019/12/2600-English-1.pdf> (visited on 24/03/2026).

- [9] Muddasar Naeem, Syed Tahir Hussain Rizvi and Antonio Coronato. ‘A Gentle Introduction to Reinforcement Learning and its Application in Different Fields’. eng. In: *IEEE access* 8 (2020), pp. 209320–209344. ISSN: 2169-3536.
- [10] Diego Perez-Liebana, Jialin Liu, Ahmed Khalifa, Raluca D. Gaina, Julian Togelius and Simon M. Lucas. ‘General Video Game AI: A Multitrack Framework for Evaluating Agents, Games, and Content Generation Algorithms’. In: *IEEE Transactions on Games* 11.3 (2019), pp. 195–214. DOI: 10.1109/TG.2019.2901021. URL: <https://doi.org/10.1109/TG.2019.2901021>.
- [11] Radisa H. Rachmadi, Rainamira Azzahra, Rayhan A. Darmawan, Patrick A. Nigo and Nunung N. Qomariyah. ‘Developing AI Bots with Minimax Algorithm for Surakarta Board Game’. In: *2021 International Conference on ICT for Smart Society (ICISS)*. 2021, pp. 1–6. DOI: 10.1109/ICISS53185.2021.9533206.
- [12] Alan M. Turing. ‘Computing Machinery and Intelligence’. In: *Mind* 59.236 (1950), pp. 433–460.
- [13] Muhtar Çağkan Uludağlı and Kaya Oğuz. ‘Non-player character decision-making in computer games’. In: *Artificial Intelligence Review* 56 (2023), pp. 14159–14191. DOI: 10.1007/s10462-023-10491-7. URL: <https://doi.org/10.1007/s10462-023-10491-7>.

Appendices

Technical documentation

List of abbreviations and symbols

AI Artificial Intelligence

NPC Non-Player Characters

FSM Finite State Machine

BT Behavior Tree

RBS Rule-Base System

MCTS Monte Carlo Tree Search

List of additional files in electronic submission (if applicable)

Additional files uploaded to the system include:

- source code of the application,
- test data,
- a video file showing how software or hardware developed for thesis is used,
- etc.

List of Figures

3.1	Figure caption.	15
-----	-------------------------	----

List of Tables

3.1	A caption of a table is ABOVE it.	16
-----	---	----